



PROYECTO COMPUTACIÓN I

ANTEPROYECTO

PROFESORA

ISABEL SUTIL MARTIN

GRUPO 2

José Fernández Tébar

Gino Milthon Levano Carhuancho

Javier Belloch Molinero

Iván Sanz López

Miguel Ángel Carpio Cornejo

Sistema de Predicción y Despacho Energético mediante Machine Learning

Contexto

El proyecto se integra en el Gemelo Digital de la isla de Aruba y se centra en el componente energético. El documento base plantea que la red necesita anticipar variaciones de demanda y de generación renovable, especialmente eólica, para disponer de margen de actuación antes de que aparezca un déficit de potencia. Como referencia del contexto real, WEB Aruba indica que el parque eólico Vader Piet está formado por 10 aerogeneradores de 3 MW (30 MW en total) y aporta una fracción relevante de la producción eléctrica insular. IRENA también refleja la elevada dependencia de Aruba de combustibles fósiles en su suministro energético y el peso de la eólica dentro de sus renovables.

Problema y/o necesidad:

La generación eólica varía con rapidez y de forma poco predecible a corto plazo: una bajada de viento puede reducir la producción del parque en pocos minutos. Como la red no puede importar energía del exterior, cualquier caída tiene que cubrirse de inmediato con generación local de respaldo. Ese respaldo, sin embargo, no responde de forma instantánea, porque los grupos térmicos necesitan tiempo de arranque y sincronización con la red.

Sin una herramienta de anticipación, la operación de la red queda atrapada entre dos alternativas insatisfactorias. La primera es actuar de forma reactiva, cuando el déficit ya se ha producido, con el riesgo de caídas de tensión o apagones. La segunda es mantener la reserva térmica en funcionamiento de forma preventiva y continua, lo que supone más consumo de combustible, más emisiones y un peor aprovechamiento de la energía eólica disponible.

A esto se suma una limitación de partida: los datos disponibles recogen únicamente variables meteorológicas registradas por las estaciones de la isla, sin mediciones directas de potencia generada. La relación entre las condiciones meteorológicas y la

potencia que producirá el parque en los minutos siguientes no está modelada y, por tanto, no se aprovecha para la toma de decisiones.

Solución Propuesta:

Para anticipar las caídas de generación eólica, se propone un sistema de predicción basado en aprendizaje automático supervisado. El sistema estima la potencia del parque con 15 minutos de antelación, la compara con la demanda esperada y traduce el resultado en una recomendación de despacho. El horizonte de 15 minutos da margen para activar el respaldo antes de que aparezca el déficit y mantiene, a la vez, una fiabilidad razonable en la predicción.

Además de las lecturas de las estaciones meteorológicas, la solución incorpora tres fuentes de datos complementarias:

- **Datos meteorológicos de reanálisis** (ERA5 u Open-Meteo) para las coordenadas de Aruba. Aportan la velocidad y dirección del viento a 100 metros, la altura aproximada del buje de los aerogeneradores, y la radiación solar. Las estaciones miden el viento cerca del suelo, mientras que los aerogeneradores lo reciben a mayor altura.
- **Especificaciones técnicas de los aerogeneradores del parque**, en particular su curva de potencia y sus velocidades de arranque, nominal y corte, tomadas de las fichas del fabricante. Sustituyen a una curva genérica y ajustan la estimación a las turbinas reales.
- **Datos de demanda eléctrica de la isla**. Permiten pasar de predecir solo la generación a estimar el balance entre generación y consumo, es decir, el déficit que debe cubrir el respaldo.

El sistema se organiza en cinco etapas:

1. Ingesta, integración y depuración. Se cargan las lecturas de las estaciones y se cruzan con su ubicación y estado operativo. Los datos de reanálisis y de demanda se alinean temporalmente con ellas: como suelen tener resolución horaria, se interpolan a una rejilla regular de 5 minutos. Después se ordena la serie por estación y marca de tiempo y se tratan los duplicados, las inconsistencias de formato y los valores anómalos.

2. Construcción de las variables objetivo. La potencia eólica se estima aplicando la curva de potencia real de los aerogeneradores al viento a altura de buje. Ese viento se obtiene del reanálisis, corregido con las mediciones de la estación más cercana, o extrapolado desde la altura de medición con un perfil vertical de viento. La potencia de una turbina se escala al conjunto del parque. Con la demanda y la estimación de generación renovable se calcula el déficit previsto.

3. Ingeniería de características. Se generan valores retardados de potencia, viento y demanda a 5, 10, 15 y 20 minutos, que recogen la tendencia reciente. Se completan con estadísticos de ventana (media, desviación y variación), con la radiación solar y con variables temporales como la hora del día y el día de la semana, que explican buena parte del patrón de consumo. Las variables objetivo se desplazan 15 minutos hacia el futuro, y las filas sin valores válidos se eliminan.

4. Entrenamiento, evaluación y definición de umbrales. Los datos se dividen de forma cronológica: 80 % para entrenamiento y 20 % para evaluación. Se entrenan varios modelos de regresión y se comparan con un modelo de referencia de persistencia, para comprobar que aportan una mejora real. Se evalúan con RMSE y R^2 , y el mejor modelo se exporta como *modelo_aruba_energia.pkl*. Los umbrales se fijan a partir del histórico con dos criterios:

- Alertas por caída de generación eólica: el percentil 15 de la potencia y porcentajes de la capacidad nominal distinguen entre operación estable, respaldo moderado (solar o baterías) y respaldo térmico crítico. La radiación solar indica si el respaldo moderado es viable en ese momento.

- Alertas por sobrecarga: se activan cuando la demanda prevista supera la capacidad renovable disponible.

5. Inferencia y visualización. El modelo se integra en dos herramientas:

- Un script *predict.py*, que recibe un registro de entrada, calcula la predicción a $t+15$ minutos y emite por consola el estado de la red.
- Una aplicación web en Streamlit, que muestra la generación prevista, la demanda y el déficit, y lanza una alerta visual según el umbral superado.

Alcance:

El proyecto se centra en el sistema eléctrico de la isla de Aruba y, en concreto, en el parque eólico de Vader Piet como fuente de generación a predecir. El sistema trabaja con un horizonte de predicción de 15 minutos y una resolución temporal de 5 minutos, sobre el periodo que cubren los datos de las estaciones meteorológicas. Los datos externos se limitan a ese mismo periodo y a la zona geográfica de la isla.

El resultado es un prototipo funcional de carácter académico, pensado como módulo del Gemelo Digital. Su usuario objetivo es el operador de la red, al que el sistema ofrece predicciones y recomendaciones de despacho como apoyo a la decisión, sin actuar por sí mismo sobre la infraestructura. El prototipo funciona en local o en un entorno de demostración, con datos históricos y registros introducidos o cargados por el usuario.

El proyecto se desarrolla siguiendo la estructura de un Trabajo Fin de Grado, con un plan de trabajo inicial, dos entregas intermedias (aproximadamente al 25 % y al 60 % del avance), una entrega final con la memoria completa y una presentación en clase. En cada entrega se actualizan el código, los modelos, los datos procesados y la documentación en el repositorio GitHub del grupo, al que el profesorado tiene acceso durante todo el desarrollo.

La validez de los resultados queda condicionada por varios supuestos. La generación del parque se estima a partir de la

curva de potencia y de datos meteorológicos, sin mediciones reales de producción, por lo que no se reflejan pérdidas por estela, indisponibilidad de turbinas ni paradas por mantenimiento. Los datos de reanálisis tienen menor resolución espacial y temporal que las estaciones, y su interpolación introduce cierta incertidumbre. Por último, la precisión de las alertas por sobrecarga depende de la resolución y representatividad de los datos de demanda disponibles.

Requisitos:

Alta => Sin esto, el proyecto no cumple el objetivo principal o un requisito solicitado por la profesora.

Media => Importante y debería estar, pero el sistema básico podría funcionar sin ello.

Baja => Mejora o complemento; no impide demostrar el funcionamiento principal.

Requisitos Funcionales:

ID	NOMBRE	DESCRIPCIÓN	PRIORIDAD
RF-01	Ingesta de datos de estaciones	El sistema deberá cargar los ficheros <i>aruba_lecturas.json</i> y <i>aruba_stations.json</i> y relacionar cada lectura con su estación mediante <i>station_id</i> .	Alta
RF-02	Integración de fuentes externas	El sistema deberá incorporar los datos de reanálisis meteorológico, las especificaciones técnicas de los aerogeneradores y los datos de demanda eléctrica, alineados con las lecturas de las estaciones.	Media
RF-03	Limpieza y validación de datos	El sistema deberá ordenar la serie temporal de forma ascendente por <i>timestamp</i> , eliminar lecturas duplicadas, corregir inconsistencias de tipo y formato, y detectar y tratar valores anómalos.	Alta
RF-04	Alineación temporal	El sistema deberá remuestrear las series de cada estación y de las fuentes externas a una rejilla regular de 5 minutos.	Media
RF-05	Estimación de potencia eólica	El sistema deberá calcular la potencia del parque aplicando la curva de potencia de los aerogeneradores a la velocidad del viento a altura de buje, expresada en m/s.	Media
RF-06	Generación de características retardadas	El sistema deberá generar, por estación, variables retardadas de potencia, viento y demanda a t-5, t-10, t-15 y t-20 minutos, junto con estadísticos de ventana (media, desviación y variación).	Alta
RF-07	Generación de	El sistema deberá crear las variables objetivo desplazando la potencia 5,	Alta

	variables objetivo	10, 15 y 20 minutos hacia el futuro, siendo t+15 el horizonte principal, y eliminar las filas con valores nulos resultantes.	
RF-08	Cálculo de umbrales operativos	El sistema deberá calcular los umbrales de alerta a partir del histórico (percentil 15 de la potencia y porcentajes de la capacidad nominal) y el umbral de sobrecarga por demanda.	Alta
RF-09	División temporal de datos	El sistema deberá dividir el dataset respetando el orden cronológico: 80% para entrenamiento y 20% para evaluación.	Alta
RF-10	Entrenamiento de modelos	El sistema deberá entrenar al menos dos modelos de regresión y un modelo de referencia de persistencia.	Alta
RF-11	Evaluación de modelos	El sistema deberá calcular el RMSE y el R^2 de cada modelo sobre el conjunto de evaluación y seleccionar el de mejor rendimiento.	Alta
RF-12	Exportación del modelo	El sistema deberá guardar el modelo seleccionado en la raíz del proyecto con el nombre <i>modelo_aruba_energia.pkl</i> .	Alta
RF-13	Inferencia por consola	El script <i>predict.py</i> deberá cargar el modelo, recibir un vector de entrada, predecir la potencia a t+15 minutos y mostrar "[ALERTA] Despacho Térmico/Solar requerido en 15m" si es inferior a 500 kW, u "[OK] Operación Estable" en caso contrario.	Alta
RF-14	Carga del modelo en la interfaz	La aplicación Streamlit deberá cargar el fichero <i>modelo_aruba_energia.pkl</i> al iniciarse.	Alta
RF-15	Entrada de registros	La aplicación deberá permitir introducir un registro manualmente mediante formulario o cargarlo desde un fichero JSON o CSV .	Alta

RF-16	Validación de entradas	La aplicación deberá comprobar que los valores introducidos están completos y dentro de rangos físicos válidos, e informar al usuario de los errores.	Media
RF-17	Visualización de la predicción	La aplicación deberá mostrar la potencia prevista a t+15 minutos, la demanda esperada y el déficit resultante.	Alta
RF-18	Alerta visual por niveles	La aplicación deberá mostrar una alerta diferenciada según el nivel alcanzado: operación estable, respaldo moderado, respaldo térmico crítico o sobrecarga por demanda.	Media
RF-19	Visualización de tendencia	La aplicación deberá mostrar una gráfica con la evolución reciente de la potencia y la predicción.	Baja

Requisitos No Funcionales:

ID	REQUISITO	TIPO
RNF-01	El sistema deberá generar la predicción de un registro en un tiempo máximo de 2 segundos.	Eficiencia
RNF-02	El proceso de entrenamiento deberá poder ejecutarse completo en un equipo personal con 8 GB de RAM .	Eficiencia
RNF-03	Ante una entrada inválida o la ausencia del modelo <i>.pkl</i> , el sistema deberá mostrar un mensaje de error comprensible sin interrumpir su funcionamiento.	Fiabilidad
RNF-04	El entrenamiento deberá ser reproducible, obteniendo las mismas métricas con las mismas semillas aleatorias y versiones de librerías.	Fiabilidad
RNF-05	La interfaz deberá permitir a un operador	Usabilidad

	sin conocimientos de aprendizaje automático obtener una predicción sin ayuda externa.	
RNF-06	Cada nivel de alerta deberá identificarse mediante color, texto e icono, sin depender únicamente del color.	Usabilidad
RNF-07	El código deberá organizarse en módulos independientes para datos, características, modelo e interfaz.	Mantenibilidad
RNF-08	Los umbrales y parámetros del sistema deberán definirse en un fichero de configuración, de modo que se puedan modificar sin cambiar el código fuente.	Mantenibilidad
RNF-09	El sistema deberá poder instalarse y ejecutarse en Windows, Linux y macOS con Python 3.10 o superior a partir del fichero <i>requirements.txt</i> .	Portabilidad
RNF-10	Las versiones de las librerías usadas en el entrenamiento deberán fijarse en <i>requirements.txt</i> para garantizar que el modelo exportado se carga correctamente en inferencia.	Compatibilidad
RNF-11	El repositorio no deberá contener credenciales ni claves de acceso; estas se gestionarán mediante variables de entorno.	Seguridad
RNF-12	La aplicación solo deberá cargar el modelo serializado generado por el propio proyecto, sin aceptar ficheros <i>.pkl</i> subidos por el usuario.	Seguridad

Constrains (Restricciones):

ID	DESCRIPCIÓN	RELACIONADO CON
RD-01	El sistema deberá desarrollarse en Python, con la interfaz de usuario en Streamlit.	Enunciado
RD-02	El modelo deberá serializarse en formato <i>.pkl</i> con el nombre <i>modelo_aruba_energia.pkl</i>	Enunciado
RD-03	No se permite usar validación cruzada K-Fold aleatoria que no respete el orden temporal.	Enunciado
RD-04	El código, los modelos, los datos y la memoria deberán alojarse en un repositorio GitHub del grupo con acceso permanente del profesorado.	Enunciado

Objetivo General:

Desarrollar un sistema de predicción de potencia eólica basado en aprendizaje automático, integrado en el Gemelo Digital de Aruba, que anticipe con 15 minutos de antelación la generación del parque y emita alertas de despacho energético que apoyen al operador en la activación preventiva de fuentes de respaldo.

Objetivos Específicos:

OE-01. Analizar y depurar los datos de las estaciones meteorológicas de Aruba, identificando su frecuencia de muestreo, duplicados, inconsistencias y valores anómalos.

OE-02. Integrar fuentes de datos complementarias (reanálisis meteorológico, especificaciones técnicas de los aerogeneradores y demanda eléctrica) en un dataset unificado con una resolución temporal de 5 minutos.

OE-03. Estimar la potencia generada por el parque eólico a partir de la velocidad del viento a altura de buje y la curva de potencia de sus aerogeneradores.

OE-04. Diseñar un conjunto de características temporales, con variables retardadas a 5, 10, 15 y 20 minutos y estadísticos de ventana, que recoja la tendencia de la serie.

OE-05. Definir umbrales operativos de alerta, justificados estadísticamente a partir del histórico, para distinguir entre operación estable, respaldo moderado, respaldo térmico crítico y sobrecarga por demanda.

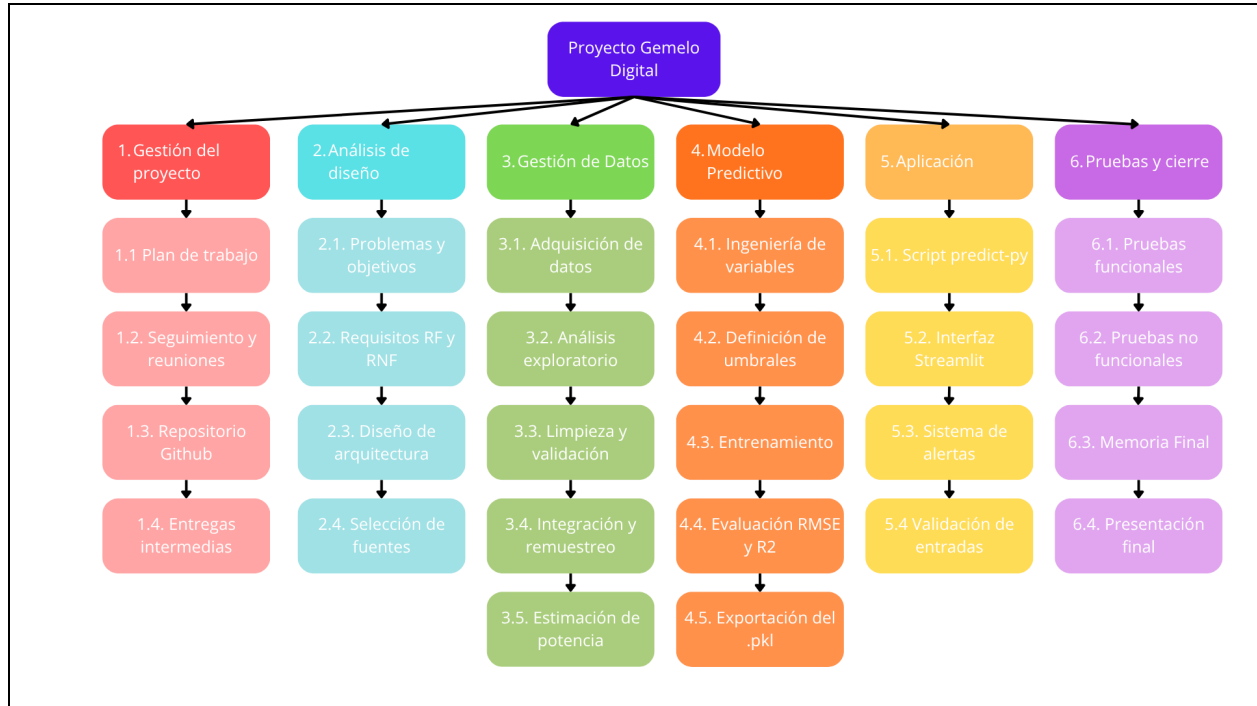
OE-06. Entrenar y comparar varios modelos de regresión con validación cronológica y seleccionar el que obtenga mejores valores de RMSE y R^2 , superando al modelo de referencia de persistencia.

OE-07. Exportar el modelo seleccionado en formato `.pkl` y desarrollar un script de inferencia por consola que evalúe la predicción a $t+15$ minutos frente a los umbrales definidos.

OE-08. Desarrollar una aplicación web en Streamlit que permita introducir o cargar registros, visualizar la predicción, la demanda y el déficit, y mostrar alertas visuales según el nivel de riesgo.

OE-09. Documentar el proceso de desarrollo y los resultados en una memoria técnica y gestionar el código, los modelos y los datos en un repositorio GitHub del grupo.

Estructura de Desglose del Trabajo:



Link canva => <https://canva.link/q571elttj9dwga4>

Desglose de tareas:

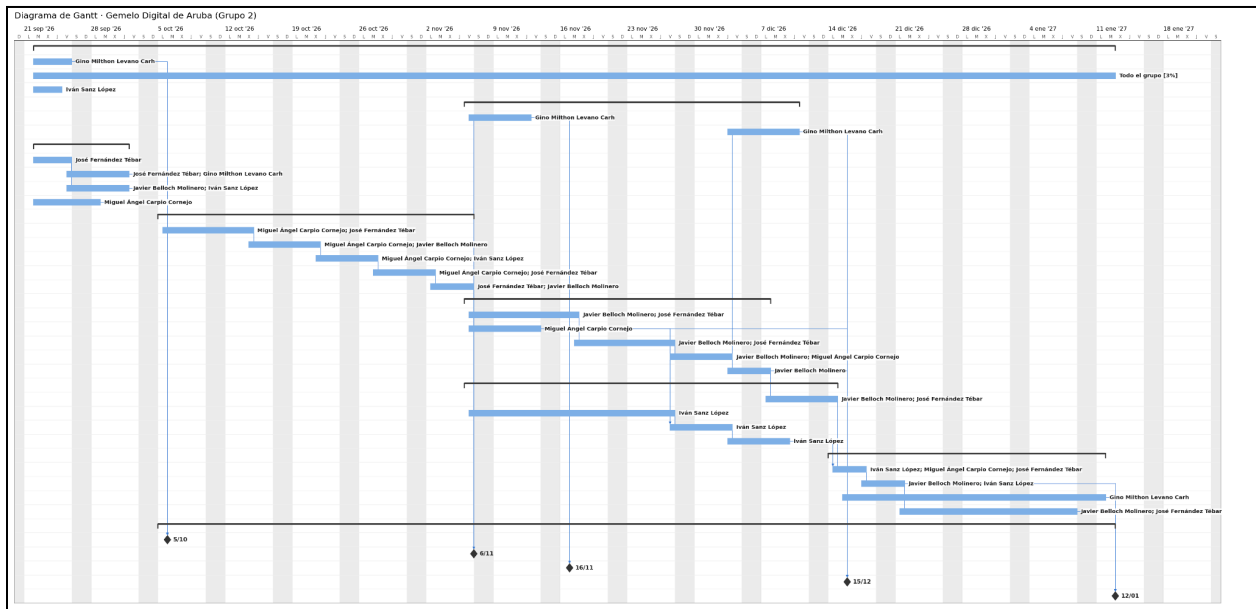
Código	Paquete de trabajo	Descripción	Entregable	Trazabilidad
1	Gestión del proyecto	Planificación, coordinación y control del proyecto		OE-09
1.1	Plan de trabajo	Definir las tareas, responsables, dependencias y calendario del proyecto	Plan de trabajo	OE-09
1.2	Seguimiento y reuniones	Revisar el avance del grupo, registrar decisiones y replanificar si hay desviaciones	Actas de seguimiento	OE-09
1.3	Repositorio Github	Crear y mantener el repositorio con su estructura, ramas y acceso del profesorado	Repositorio del grupo	RD-04

1.4	Entregas intermedias	Preparar y entregar el proyecto y la memoria al 25 % y al 60 % de avance	Entrega 1 y entrega 2	OE-09
2	Análisis y diseño	Definición del problema, los requisitos y la arquitectura del sistema		OE-01, OE-02
2.1	Problema y objetivos	Redactar el contexto, el problema, la solución propuesta, el alcance y los objetivos	Capítulo inicial de la memoria	—
2.2	Requisitos RF y RNF	Especificar los requisitos funcionales, no funcionales y las restricciones de diseño	Especificación de requisitos	RF-01 a RF-19, RNF-01 a RNF-14
2.3	Diseño de arquitectura	Diseñar los módulos del sistema, el flujo de datos y la estructura del código	Documento de arquitectura	RNF-08
2.4	Selección de fuentes	Elegir y justificar las fuentes de reanálisis, las especificaciones de aerogeneradores y los datos de demanda	Inventario de fuentes de datos	OE-02
3	Gestión de datos	Obtención, depuración e integración de los datos del proyecto		OE-01 a OE-03
3.1	Adquisición de datos	Descargar los ficheros de la asignatura y las fuentes externas seleccionadas	Datos en bruto	RF-01, RF-02
3.2	Análisis exploratorio	Estudiar la frecuencia de muestreo, las estaciones, los rangos y la distribución de las variables	Informe EDA	OE-01

3.3	Limpieza y validación	Ordenar la serie y tratar duplicados, inconsistencias y anomalías	Dataset depurado	RF-03
3.4	Integración y remuestreo	Unir todas las fuentes y alinearlas en una rejilla de 5 minutos	Dataset integrado	RF-02, RF-04
3.5	Estimación de potencia	Calcular la potencia del parque con la curva de potencia y el viento a altura de buje	Serie de potencia estimada	RF-05
4	Modelado predictivo	Construcción, evaluación y exportación del modelo de predicción		OE-04 a OE-07
4.1	Ingeniería de variables	Generar variables retardadas, estadísticos de ventana, variables temporales y objetivos	Dataset de entrenamiento	RF-06, RF-07
4.2	Definición de umbrales	Calcular y justificar los umbrales de alerta a partir del histórico	Tabla de umbrales	RF-08
4.3	Entrenamiento	Dividir los datos de forma cronológica y entrenar los modelos y el de persistencia	Modelos entrenados	RF-09, RF-10
4.4	Evaluación RMSE y R^2	Comparar los modelos, analizar errores y seleccionar el mejor	Informe de resultados	RF-11, RNF-01
4.5	Exportación del .pkl	Serializar el modelo seleccionado en la raíz del proyecto	modelo_aruba_energia.pkl	RF-12
5	Aplicación	Desarrollo de las herramientas de inferencia y visualización		OE-07, OE-08

5.1	Script predict.py	Implementar la inferencia por consola con los mensajes de estado de la red	predict.py	RF-13
5.2	Interfaz Streamlit	Desarrollar la aplicación web con carga del modelo, entrada de registros y visualización	Aplicación Streamlit	RF-14, RF-15, RF-17, RF-19
5.3	Sistema de alertas	Implementar las alertas visuales por niveles según los umbrales definidos	Módulo de alertas	RF-18, RNF-07
5.4	Validación de entradas	Comprobar la completitud y los rangos de los datos y gestionar los errores	Módulo de validación	RF-16, RNF-04
6	Pruebas y cierre	Verificación del sistema y cierre del proyecto		OE-09
6.1	Pruebas funcionales	Verificar que el sistema cumple cada requisito funcional	Informe de pruebas funcionales	RF-01 a RF-19
6.2	Pruebas no funcionales	Medir rendimiento, portabilidad, reproducibilidad y usabilidad	Informe de pruebas no funcionales	RNF-01 a RNF-14
6.3	Memoria final	Completar y revisar la memoria técnica del proyecto	Memoria final	OE-09
6.4	Presentación final	Preparar y realizar la defensa del proyecto en clase	Presentación	OE-09

Planificación del Cronograma:



ID	Nombre de tarea	Duración	Comienzo	Fin	Predecesoras	Nombres de los recursos
1	Gestión del proyecto	80.75 días	mar 22/09/26	mar 12/01/27		
2	Plan de trabajo	3.75 días	mar 22/09/26	vie 25/09/26		Gino Milthon Levano Carh
3	Seguimiento y reuniones semanales	80.75 días	mar 22/09/26	mar 12/01/27		Gino Milthon Levano Carh; José Fernández Tébar ;Javier Belloch Molinero; Iván Sanz López; Miguel Ángel Carpio Cornejo
4	Repositorio GitHub	2.75 días	mar 22/09/26	jue 24/09/26		Iván Sanz López
5	Entregas intermedias	24 días	vie 6/11/26	jue 10/12/26		

6	Documentación Check 1	4 días	vie 6/11/26	jue 12/11/26	37	Gino Milthon Levano Carh
7	Documentación Check modelo	5 días	jue 3/12/26	jue 10/12/26	23	Gino Milthon Levano Carh
8	Análisis y diseño	7.75 días	mar 22/09/26	jue 1/10/26		
9	Problema y objetivos	3.75 días	mar 22/09/26	vie 25/09/26		José Fernández Tébar
10	Requisitos RF y RNF	4 días	vie 25/09/26	jue 1/10/26	9	José Fernández Tébar;Gino Milthon Levano Carh
11	Diseño de arquitectura	4 días	vie 25/09/26	jue 1/10/26	9	Javier Belloch Molinero;Iván Sanz López
12	Selección de fuentes	4.75 días	mar 22/09/26	lun 28/09/26		Miguel Ángel Carpio Cornejo
13	Gestión de datos	24 días	lun 5/10/26	vie 6/11/26		
14	Adquisición de datos	7 días	lun 5/10/26	mié 14/10/26	36	Miguel Ángel Carpio Cornejo;José Fernández Tébar
15	Análisis exploratorio	5 días	mié 14/10/26	mié 21/10/26	14	Miguel Ángel Carpio Cornejo;Javier Belloch Molinero
16	Limpieza y validación	4 días	mié 21/10/26	mar 27/10/26	15	Miguel Ángel Carpio Cornejo;Iván Sanz López

17	Integración y remuestreo	4 días	mar 27/10/26	lun 2/11/26	16	Miguel Ángel Carpio Cornejo; José Fernández Tébar
18	Estimación de potencia	4 días	lun 2/11/26	vie 6/11/26	17	José Fernández Tébar; Javier Belloch Molinero
19	Modelado predictivo	21 días	vie 6/11/26	lun 7/12/26		
20	Ingeniería de variables	7 días	vie 6/11/26	mar 17/11/26	37	Javier Belloch Molinero; José Fernández Tébar
21	Definición de umbrales	5 días	vie 6/11/26	vie 13/11/26	37	Miguel Ángel Carpio Cornejo
22	Entrenamiento	8 días	mar 17/11/26	vie 27/11/26	20	Javier Belloch Molinero; José Fernández Tébar
23	Evaluación RMSE y R^2	4 días	vie 27/11/26	jue 3/12/26	22	Javier Belloch Molinero; Miguel Ángel Carpio Cornejo
24	Exportación del .pkl	2 días	jue 3/12/26	lun 7/12/26	23	Javier Belloch Molinero
25	Aplicación	26 días	vie 6/11/26	lun 14/12/26		

26	Script predict.py	5 días	lun 7/12/26	lun 14/12/26	24	Javier Belloch Molinero; José Fernández Tébar
27	Interfaz Streamlit	15 días	vie 6/11/26	vie 27/11/26	37	Iván Sanz López
28	Sistema de alertas	4 días	vie 27/11/26	jue 3/12/26	27;21	Iván Sanz López
29	Validación de entradas	4 días	jue 3/12/26	mié 9/12/26	28	Iván Sanz López
30	Pruebas y cierre	20 días	lun 14/12/26	lun 11/01/27		
31	Pruebas funcionales	3 días	lun 14/12/26	jue 17/12/26	26;29	Iván Sanz López; Miguel Ángel Carpio Cornejo; José Fernández Tébar
32	Pruebas no funcionales	2 días	jue 17/12/26	lun 21/12/26	31	Javier Belloch Molinero; Iván Sanz López
33	Memoria final	19 días	mar 15/12/26	lun 11/01/27	39	Gino Milthon Levano Carh
34	Presentación final	14 días	lun 21/12/26	vie 8/01/27	32	Javier Belloch Molinero; José Fernández Tébar
35	Hitos del proyecto	71 días	lun 5/10/26	mar 12/01/27		
36	Hito: Anteproyecto	0 días	lun 5/10/26	lun 5/10/26	2;10; 11;12	
37	Hito: Entrega ETL (límite dom. 8 nov)	0 días	vie 6/11/26	vie 6/11/26	18	

38	Hito: Check 1	0 días	lun 16/11/26	lun 16/11/26	6	
39	Hito: Check modelo	0 días	mar 15/12/26	mar 15/12/26	24;21 ;7	
40	Hito: Entrega final: app y memoria	0 días	mar 12/01/27	mar 12/01/27	32;33 ;34	